

**STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ
ZOCHOVA 9, BRATISLAVA**



ProMat

projekt z praktickej časti odbornej zložky maturitnej skúšky

Meno kandidáta: Ján Dobák

Vedúci práce: Lenka Ráčková

Trieda: IV.C

Šk. rok: 2025/2026

**STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ
ZOCHOVA 9, BRATISLAVA**

ProMat

Meno kandidáta: Ján Dobák

Trieda: IV.C

Vedúci práce: Lenka Ráčková

Rozsah: 29 strán, 15 obrázkov

Odovzdané: 10.4.2026

Podpis vedúceho práce: _____

Tu bude vložený originál zadania projektu.

Čestné prehlásenie

Čestne prehlasujem, že som maturitný projekt vypracoval samostatne s použitím odbornej literatúry a zdrojov uvedených v zozname bibliografických odkazov.

V Bratislave dňa TBD

Ján Dobák

Pod'akovanie

Chcel by som sa pod'akovať svojej vedúcej práce nie len za pomoc a vedení pri tvorbe maturitného projektu ale aj za celé štyri roky, ktorý vo mňa veril a naučil ma mnoho do budúcnosti.

OBSAH

Čestné prehlásenie	4
Pod’akovanie	5
Zoznam skratiek	9
Úvod	1

TEORETICKÁ ČASŤ

1 Riešená problematika	2
1.1 Popis problému	2
1.2 Súčasné riešenia	2
1.3 Požiadavky na systém	3
2 Cieľ práce	4
3 Použité technológie	5
3.1 Backend	5
3.1.1 Python	5
3.1.2 Flask Framework	6
3.1.3 Použité knižnice a moduly	6
3.2 Frontend	7
3.2.1 HTML a Jinja2 Šablóny	7
3.2.2 JavaScript a Interaktivita	7
3.2.3 Responzívny Dizajn	7
3.3 Databáza	8
3.3.1 SQLite	8
3.4 Ostatné nástroje	8
3.4.1 AI a LLM Integrácia	8
3.4.2 Knižnice pre Dokumenty	8
3.4.3 Verzia Kontrola a Deployment	9
3.4.4 Bezpečnosť a Overovanie	9
3.4.5 Výkon a Caching	9

PRAKTICKÁ ČASŤ	9
4 Dizajn a architektúra	10
4.1 Architektúra systému	10
4.2 Štruktúra projektu	10
4.3 Používateľské rozhranie	11
5 Databázový model	13
6 Autentifikácia	14
6.1 Registrácia	14
6.2 Prihlásenie	14
6.3 Správa relácie	15
7 Projekty a úlohy	16
7.1 Správa projektov	16
7.2 Správa úloh	17
7.3 Priradenie členov	18
8 Dokumenty	19
8.1 Tvorba dokumentov	19
8.2 Zdieľanie dokumentov	20
9 AI Chat	21
9.1 Integrácia AI	21
9.2 Rozhranie chatu	21
10 Dashboard	24
10.1 Prehľad a úvodné informácie	24
10.2 AI-asistované poznatky	24
10.2.1 Typy poznatkov	24
10.3 Sledovanie prioritných projektov	25
10.4 Vyhľadávanie	25
10.5 Bezpečnosť a oprávnenia	25
Záver	27

ZOZNAM OBRÁZKOV

4.1	Dashboard aplikácie	11
4.2	Kanban tabuľka projektu	12
6.1	Registračný formulár	14
6.2	Prihlasovací formulár	15
7.1	Zoznam projektov	16
7.2	Detail projektu	17
7.3	Kanban tabuľka úloh projektu	18
7.4	Správa členov projektu	18
8.1	Nahrávanie dokumentu	19
8.2	Zoznam dokumentov projektu	20
9.1	Rozhranie AI chatu	22
9.2	Príklad AI odpovede nad obsahom dokumentu	22
10.1	Dashboard aplikácie ProMat	25
10.2	Štatistiky a AI-asistované poznatky na dashboarde	26

ZOZNAM SKRATIEK

Skratka	Popis
ACID	Atomicity, Consistency, Isolation, Durability (vlastnosti databázových transakcií)
AI	Artificial Intelligence (umelá inteligencia)
API	Application Programming Interface (rozhranie pre programovanie aplikácií)
CSS	Cascading Style Sheets (kaskádové štýly)
CSRF	Cross-Site Request Forgery (falšovanie medzistránkových požiadaviek)
DB	Database (databáza)
DOCX	Office Open XML Document (formát dokumentu Microsoft Word)
GIL	Global Interpreter Lock (globálny zámok interpretera v Pythone)
HTML	HyperText Markup Language (hypertextový značkovací jazyk)
HTTP	Hypertext Transfer Protocol (protokol pre prenos hypertextu)
HTTPS	Hypertext Transfer Protocol Secure (zabezpečený HTTP protokol)
JSON	JavaScript Object Notation (formát výmeny dát)
LLM	Large Language Model (veľký jazykový model)
NLP	Natural Language Processing (spracovanie prirodzeného jazyka)
ORM	Object-Relational Mapping (objektovo-relačné mapovanie)
PDF	Portable Document Format (prenosný formát dokumentu)
REST	Representational State Transfer (architektonický štýl webových služieb)
SQL	Structured Query Language (štruktúrovaný dopytovací jazyk)
SSL	Secure Sockets Layer (vrstva zabezpečených socketov)
TLS	Transport Layer Security (zabezpečenie transportnej vrstvy)
UI	User Interface (používateľské rozhranie)
URL	Uniform Resource Locator (jednotné označovanie zdrojov)
UUID	Universally Unique Identifier (univerzálny jedinečný identifikátor)
WSGI	Web Server Gateway Interface (rozhranie brány webového servera)

Úvod

V súčasnej praxi rastie potreba efektívne riadiť projekty, úlohy a sprievodnú dokumentáciu na jednom mieste. Mnohé tímy však stále pracujú s oddelenými nástrojmi, čo vedie k strate prehľadu, duplicite informácií a časovým stratám pri vyhľadávaní podkladov. Tento problém je výrazný najmä v oblastiach, kde sa pravidelne pracuje s rozsiahlejšími textami a formálnymi dokumentmi, napríklad pri právnickej činnosti, plánovaní táborov alebo organizovaní iných projektov.

V predkladanej práci sme sa zamerali na návrh a implementáciu systému na správu projektov, ktorý prepája riadenie úloh, dokumentov a používateľských rolí do jedného celku. Systém umožňuje nahrávanie dokumentov vo formátoch PDF a DOCX, ich organizáciu v kontexte konkrétnych projektov a tímovej spolupráce. Dôležitou súčasťou riešenia je integrácia AI chatu nad obsahom dokumentov, ktorý podporuje sumarizáciu a vyhľadávanie relevantných informácií priamo z nahraných materiálov.

Kľúčovou požiadavkou riešenia je ochrana dát a ich plná kontrola používateľom. Celý systém sme preto navrhli tak, aby dokázal bežať lokálne na serveri organizácie bez odosielania obsahu dokumentov tretím stranám. Takýto prístup zvyšuje dôveryhodnosť spracovávaných informácií a zároveň umožňuje nasadenie aj v prostrediach s prísnejšími bezpečnostnými alebo legislatívnymi nárokmi.

Cieľom práce bolo vytvoriť prakticky použiteľný nástroj, ktorý zjednoduší každodennú prácu s projektami a dokumentmi, skráti čas potrebný na orientáciu v podkladoch a podporí rozhodovanie na základe rýchlo dostupných informácií. Výsledné riešenie má preukázať, že prepojenie projektového manažmentu s inteligentnou prácou s dokumentmi môže výrazne zvýšiť efektivitu tímov v rôznych typoch činností.

TEORETICKÁ ČASŤ

1 Riešená problematika

Riadenie úloh je často oddelené od práce s dokumentmi a od bezpečného spracovania citlivých informácií. Cieľom je identifikovať, kde vznikajú najväčšie prekážky, ako ich riešia existujúce nástroje a aké požiadavky musí spĺňať navrhovaný systém.

1.1 Popis problému

V mnohých organizáciách prebieha projektové riadenie v jednom nástroji, dokumenty sa ukladajú v inom úložisku a diskusia o ich obsahu prebieha cez e-mail alebo chat. Tento spôsob práce spôsobuje:

- stratu kontextu medzi úlohami a súvisiacimi dokumentmi,
- pomalé vyhľadávanie informácií v rozsiahlych PDF a DOCX súboroch,
- zvýšené riziko chýb pri manuálnom sumarizovaní obsahu.

Problém je výrazný v oblastiach, kde sa pravidelne pracuje s textovo bohatými podkladmi, napríklad v právnickej agende alebo pri plánovaní táborov, akcií a väčších podujatí. V takýchto prípadoch je dôležité mať nielen prehľad o úlohách, ale aj možnosť rýchlo analyzovať obsah dokumentov priamo v kontexte projektu.

1.2 Súčasné riešenia

Na trhu existuje viacero nástrojov na správu projektov, ktoré dobre pokrývajú evidenciu úloh, termínov a tímovej spolupráce. Príkladom je Trello [1], ktoré je prehľadné, jednoduché na používanie a vhodné na vizuálne riadenie práce formou tabuliek.

Napriek týmto výhodám má Trello v kontexte tejto práce podstatné obmedzenie. Neponúka inteligentnú prácu s obsahom priložených dokumentov. Dokument je možné k úlohe pripojiť, ale systém štandardne neposkytuje:

- automatickú sumarizáciu textu z príloh,
- AI chat nad dokumentmi v rámci projektu alebo úlohy,
- systematické sledovanie verzií dokumentu a histórie zmien.

Používateľ tak musí obsah dokumentov študovať manuálne alebo použiť externé služby, čo znižuje plynulosť práce a môže vytvárať bezpečnostné riziko pri prenose dát mimo interné prostredie organizácie.

1.3 Požiadavky na systém

Na základe analýzy existujúcich riešení a potrieb cieľových používateľov boli stanovené nasledujúce požiadavky na navrhovaný systém:

- **Správa projektov a úloh** – systém musí umožňovať vytváranie projektov, priradovanie úloh členom tímu a sledovanie ich stavu.
- **Nahrávanie dokumentov** – podpora formátov PDF a DOCX s ukladaním na lokálny server organizácie.
- **AI chat nad dokumentmi** – možnosť klásť otázky v prirodzenom jazyku nad obsahom konkrétnych nahraných súborov.
- **Tímová spolupráca** – systém rieši používateľské roly (administrátor, manažér, člen) s rôznymi oprávneniami.
- **Bezpečnosť a lokálna prevádzka** – aplikácia musí bežať na HTTPS, neprenášať dokumenty na externé servery a podporovať správu relácií.
- **Responzívne rozhranie** – UI musí byť použiteľné na desktope aj mobilnom zariadení, s podporou tmavého režimu.

2 Cieľ práce

Cieľom tejto práce bolo navrhnuť a implementovať webovú aplikáciu na správu projektov s integrovaným AI chatom nad dokumentmi. Výsledný systém má nahradiť roztrieštenú prácu s viacerými nástrojmi a poskytnúť tímom jednotné prostredie pre riadenie úloh, ukladanie dokumentov a inteligentnú prácu s ich obsahom. Aplikáciu sme navrhli tak, aby bola nasaditeľná lokálne na infraštruktúre organizácie bez závislosti na externých cloudových službách na spracovanie dokumentov.

Na splnenie hlavného cieľa sme najprv navrhli architektúru systému ako modulárne riešenie pozostávajúce z backendu, frontendu, databázovej vrstvy a AI modulu, aby bolo možné funkcionality v budúcnosti jednoducho rozširovať. Následne sme implementovali správu projektov a úloh so zameraním na vytváranie projektov, evidenciu úloh, priradovanie zodpovedných osôb, sledovanie stavu a podporu základného tímového workflow. Súčasťou riešenia je aj správa dokumentov, ktorá umožňuje nahrávanie, ukladanie a organizáciu súborov v kontexte projektu s podporou formátov PDF a DOCX. Dôležitou časťou práce bola integrácia AI chatu nad dokumentmi, teda sprístupnenie otázok v prirodzenom jazyku nad obsahom vybraných dokumentov a vracanie relevantných odpovedí na základe extrahovaného textu. Zároveň sme zabezpečili, aby aplikácia fungovala bezpečne v lokálnej prevádzke, dokumenty ostávali v internom prostredí organizácie a komunikácia prebiehala cez zabezpečený HTTPS kanál. Posledným krokom bolo overenie funkčnosti riešenia prostredníctvom praktického testovania hlavných scenárov používania a zhodnotenia, či systém spĺňa stanovené funkčné a nefunkčné požiadavky.

Výstupom práce je funkčný prototyp webovej aplikácie, ktorý demonštruje, že prepojenie projektového riadenia, dokumentového manažmentu a AI asistencie v jednom systéme je realizovateľné aj v podmienkach lokálnej infraštruktúry.

3 Použité technológie

Aplikáciu sme postavili na modernom zásobníku technológií, ktorý pokrýva backendové spracovanie požiadaviek, frontendové zobrazenie a správu databázy. Výber technológií sme podmienili požiadavkou na lokálnu prevádzku, jednoduchosť nasadenia a dostupnosť kvalitných knižníc pre AI integráciu.

3.1 Backend

Backendovú vrstvu aplikácie sme implementovali v jazyku Python s využitím microframeworku Flask. Táto kombinácia umožňuje rýchly vývoj, prehľadnú štruktúru kódu a jednoduchú integráciu s knižnicami pre spracovanie dokumentov a umelú inteligenciu.

3.1.1 Python

Python je vysokoúrovňový, interpretovaný programovací jazyk známy svojou jednoduchosťou a čitateľnosťou kódu [2]. Bol vytvorený v roku 1991 Guidom van Rossum a v súčasnosti je jedným z najpopulárnejších jazykov na svete. Python vyniká minimalistickým syntaxom a obrovským ekosystémom knižníc pre rôzne oblasti aplikácií od webových aplikácií cez dátovú analýzu až po umelú inteligenciu a strojové učenie.

V kontexte webových aplikácií Python ponúka mocné frameworky ako Django a Flask, ktoré umožňujú rýchly vývoj robustných backendových riešení. Python je tiež populárny v oblasti NLP (Natural Language Processing), čím je ideálny pre náš projekt využívajúci AI chatbot.

Výhody Pythonu:

- Jednoduchá a čitateľná syntax – Python kód je ľahko zrozumiteľný, čo urýchľuje vývoj
- Obrovský ekosystém knižníc – pre takmer akúkoľvek úlohu existuje kvalitná knižnica
- Výkon v AI/ML – najobľúbenejší jazyk pre machine learning a NLP aplikácie
- Aktívna komunita – veľké množstvo tutoriálov, dokumentácie a podpory

Nevýhody Pythonu:

- Pomalší ako kompilované jazyky – interpretovaný jazyk má nižší výkon ako nízkoúrovňové jazyky ako C++ alebo Java
- Väčšia spotreba pamäte – dynamické typovanie a runtime overhead
- GIL (Global Interpreter Lock) – v multi-threaded aplikáciách limituje paralelizmus

Pre náš projekt boli výhody Pythonu, najmä jednoduchosť a vhodnosť pre AI integráciu, podstatne dôležitejšie ako nevýhody.

3.1.2 Flask Framework

Flask je ľahký a flexibilný Python web framework, ktorý je ideálny pre vývoj webových aplikácií [3]. Na rozdiel od Django, ktorý je *all-in-one* riešenie, ponúka Flask minimalistický prístup, ktorý umožňuje vývojárom zvoliť si vlastné knižnice a komponenty.

Flask je postavený na Werkzeug (WSGI toolkit) [4] a Jinja2 (template engine) [5] a poskytuje základné funkcionality ako:

- Routing – jednoduché definovanie URL ciest
- Request/Response handling – spracovanie bezpečných HTTPS požiadaviek
- Session management – správa používateľských relácií s HTTPS cookies
- Error handling – ošetrovanie chýb

V našom projekte sme Flask zvolili pre jeho jednoduchosť, flexibilitu a možnosť ľahkej integrácie s rôznymi knižnicami pre AI a databázové operácie. Aplikácia beží na HTTPS s SSL/TLS šifrovaním pre zabezpečenú komunikáciu.

3.1.3 Použité knižnice a moduly

V projekte sme využili viaceré Python knižnice:

- **SQLAlchemy** [6] – ORM (Object-Relational Mapping) knižnica pre komunikáciu s databázou
- **Flask-SQLAlchemy** [6] – integrácia SQLAlchemy s Flask
- **Flask-Login** [7] – spravovanie používateľských relácií a autentifikácie
- **Werkzeug** [4] – WSGI toolkit, ktorý Flask používa na spracovanie HTTPS požiadaviek s SSL/TLS podporou
- **Flask-WTF** [8] – CSRF ochrana vo formulároch
- **Flask-Caching** [9] – in-memory cache pre zrýchlenie opakovaných požiadaviek
- **ollama** [10] – integrácia s lokálnym LLM serverom pre AI chat funkcionality
- **google-genai** [11] – integrácia s Google Generative AI API pre náročnejšie úlohy
- **python-docx** [12] – čítanie a písanie Word dokumentov (.docx)
- **PyPDF2** [13] – spracovanie PDF súborov
- **python-dotenv** [14] – správa citlivých informácií (API kľúče, databázové pripojenia)
- **Jinja2** [5] – template engine pre dynamické HTML stránky

Tieto knižnice tvoria základ backendovej infraštruktúry a umožňujú efektívne spracovávanie požiadaviek, správu dát a integráciu AI funkcionalít.

3.2 Frontend

Frontend aplikácie sme postavili na moderných webových technológiách, ktoré zabezpečujú interaktívne a responzívne používateľské rozhranie. Hlavným zámerom bolo vytvoriť používateľsky intuitívne rozhranie s podporou tmavého režimu.

3.2.1 HTML a Jinja2 Šablóny

HTML tvorí kostru aplikácie a je dynamicky generovaný pomocou Jinja2 template engine-a. Jinja2 je mocný template engine pre Python Flask, ktorý umožňuje:

- Dynamické vkladanie dát z backendu do HTML stránok
- Opakujúce sa bloky kódu – `for` cykly a `if` výrazy
- Dedičnosť šablón – `extends` a `block` direktívy

Aplikáciu sme navrhli s modulárnou štruktúrou šablón s hlavným `base.html` súborom, z ktorého dedia všetky ďalšie šablóny. Komponenty ako header, sidebar, modálne dialógy a karty sme oddelili do samostatných šablón pre lepšiu udržiavateľnosť.

3.2.2 JavaScript a Interaktivita

JavaScript zabezpečuje dynamické správanie aplikácie bez nutnosti obnovy celej stránky. Kľúčové JavaScript moduly, ktoré sme vytvorili:

- **main.js** – základná logika: dark mode, sidebar, event listeners
- **chat.js** – interaktívny AI chat s real-time update-mi
- **kanban.js** – drag-and-drop funkcionálna pre Kanban tabuľu
- **prefetch.js** – prefetching stránky pre rýchlejšiu navigáciu

Implementované funkcionality:

- Dark mode – prepínanie medzi svetlým a tmavým režimom s uložením v `localStorage`
- Sidebar navigácia – responzívny sidebar s overlay na mobilných zariadeniach
- Živá notifikácia – dynamické aktualizácie bez obnovenia strany
- Validácia formulárov – klientska aj serverová validácia

3.2.3 Responzívny Dizajn

Aplikáciu sme navrhli ako úplne responzívnu, optimalizovanú pre rôzne veľkosti obrazovky:

- **Mobilné zariadenia** (<768 px) – sidebar sa skrýva, navigácia je zložená
- **Tablety** (768 px – 1024 px) – upravené rozloženie s menej priestorom
- **Desktopy** (>1024 px) – plné rozloženie so všetkými funkciami

3.3 Databáza

V aplikácii sme použili relačnú databázu pre trvalé uloženie všetkých dát. Model databázy sme definovali pomocou SQLAlchemy ORM, ktorý umožňuje pracovať s databázovými tabuľkami ako s Python objektmi.

3.3.1 SQLite

Ako primárny databázový systém sme zvolili SQLite [15]. SQLite je ľahký, spoľahlivý a vhodný pre aplikácie s malými až strednými dátovými objemami. Nevyžaduje samostatný databázový server – celá databáza je uložená v súbore `promat.db`.

Výhody SQLite:

- Bez potreby externého servera – databáza je jednoduchý súbor
- Jednoduchá inštalácia a konfigurácia
- Dostatočný výkon pre väčšinu prípadov
- ACID transakcie zabezpečujúce integritu dát

3.4 Ostatné nástroje

Okrem hlavných technológií sme využili niekoľko ďalších nástrojov a služieb pre rozšírenie funkcionality.

3.4.1 AI a LLM Integrácia

Aplikáciu sme integrovali s viacerými AI modelmi:

- **Ollama** [10] – lokálny LLM server s open-source modelmi (napr. Gemma 4)
- **Google Generative AI** [11] – cloudové AI API pre náročnejšie úlohy

Tieto integrácie umožňujú AI chat funkcionality a automatické generovanie informácií. Predvolený model je `gemma4 : 2 6b` [16] bežiaci lokálne, čo zabezpečuje plnú súkromnosť spracovávaných dokumentov.

3.4.2 Knižnice pre Dokumenty

Pre spracovanie dokumentov sme použili:

- **python-docx** [12] – čítanie a písanie Word dokumentov (.docx)
- **PyPDF2** [13] – spracovanie PDF súborov

Pomocou týchto knižníc aplikácia vie extrahovať text z dokumentov a umožniť AI modelom pracovať s ich obsahom.

3.4.3 Verzia Kontrola a Deployment

Na verzionovanie kódu a správu vývojového procesu sme použili Git [17]. Každú funkčnú zmenu sme zaznamenali v samostatnom commite s popisom vykonaných úprav.

3.4.4 Bezpečnosť a Overovanie

- **Flask-Login** [7] – session management a autentifikácia
- **Flask-WTF** [8] – CSRF ochrana vo formulároch
- **python-dotenv** [14] – správa citlivých informácií (API keys, databázové pripojenia)

3.4.5 Výkon a Caching

- **Flask-Caching** [9] – in-memory cache pre zrýchlenie opakovaných požiadaviek s predvoleným časovým limitom 300 sekúnd

PRAKTICKÁ ČASŤ

4 Dizajn a architektúra

V tejto kapitole opisujeme celkovú architektúru systému, štruktúru zdrojového kódu a návrh používateľského rozhrania. Správny architektonický návrh je základom udržiavateľnej a rozšíriteľnej aplikácie.

4.1 Architektúra systému

Aplikáciu sme navrhli ako klasickú klient-server webovú aplikáciu s trojvrstvovou architektúrou:

- **Prezentačná vrstva** – HTML/CSS/JavaScript šablóny (Jinja2 [5]), ktoré renderuje Flask [3] a zasiela klientovi cez HTTPS.
- **Aplikačná vrstva** – Flask blueprinty obsahujúce business logiku: spracovanie požiadaviek, autorizáciu, komunikáciu s AI modelom a extrakciu textu z dokumentov.
- **Dátová vrstva** – SQLAlchemy [6] ORM modely komunikujúce s SQLite [15] databázou. Súbory dokumentov sú uložené na filesysteme servera.

Kód sme rozdelili do samostatných Flask blueprintov podľa funkčných celkov: `auth`, `dashboard`, `projects`, `tasks`, `documents`, `ai_chat`, `team` a `settings`. Každý blueprint obsahuje vlastné routes, šablóny a pomocné funkcie, čo umožňuje nezávislý vývoj a testovanie jednotlivých modulov.

4.2 Štruktúra projektu

Zdrojový kód aplikácie sme organizovali nasledovne:

- `app/` – hlavný adresár aplikácie
 - `blueprints/` – jednotlivé moduly (`auth`, `projects`, `tasks`, ...)
 - `models/` – SQLAlchemy modely databázových entít
 - `services/` – servisná vrstva (extrakcia textu, AI komunikácia)
 - `templates/` – Jinja2 HTML šablóny
 - `static/` – CSS, JavaScript, obrázky
- `config.py` – konfigurácia aplikácie (databáza, AI, bezpečnosť)
- `run.py` – vstupný bod pre spustenie servera
- `requirements.txt` – zoznam Python závislostí
- `uploads/` – adresár pre nahrané dokumenty
- `instance/promat.db` – SQLite databázový súbor

4.3 Používateľské rozhranie

Používateľské rozhranie sme navrhli s dôrazom na jednoduchosť a prehľadnosť. Každá stránka obsahuje fixný bočný panel (sidebar) s navigáciou a hlavný obsahový priestor. Aplikácia podporuje svetlý aj tmavý režim, pričom používateľova voľba je uložená lokálne.

Rozhranie obsahuje nasledujúce kľúčové obrazovky:

- **Dashboard** – prehľad projektov, úloh a aktivity (obr. 4.1)
- **Zoznam projektov** – prehľad všetkých projektov s filtráciou
- **Detail projektu** – úlohy, dokumenty a členovia projektu
- **Kanban tabuľa** – drag-and-drop zobrazenie úloh podľa stavu (obr. 4.2)
- **Dokumenty** – nahrávanie, prezeranie a AI chat nad dokumentmi
- **AI chat** – konverzačné rozhranie pre prácu s dokumentmi

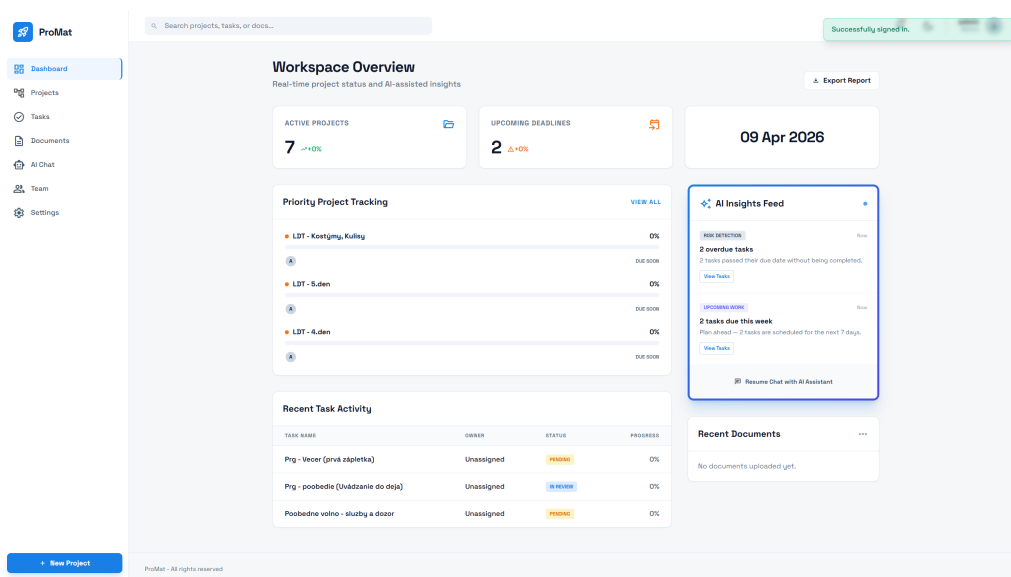


Figure 4.1 – Dashboard aplikácie

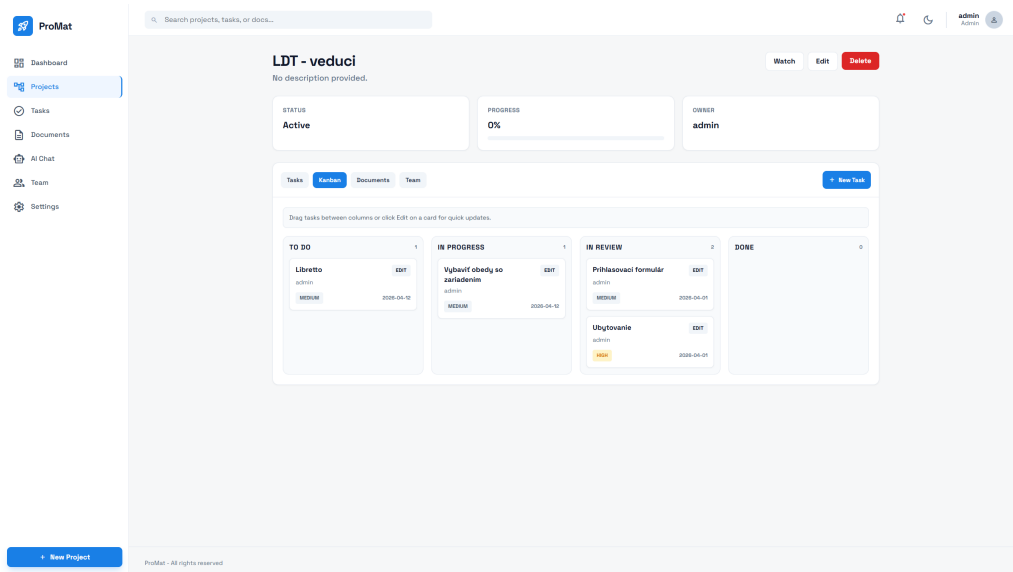


Figure 4.2 – Kanban tabuľka projektu

5 Databázový model

Databázový model tvorí jadro aplikácie a určuje štruktúru dát, väzby medzi entitami aj pravidlá integrity. Implementovali sme ho v SQLAlchemy ORM a namapovali na tabuľky SQLite. Pri návrhu sme vychádzali z potreby prepojiť projektové riadenie, správu dokumentov a AI komunikáciu v jednom konzistentnom priestore bez zbytočnej duplicity.

Centrálna je entita projektu, na ktorú sa viažu procesné dáta (úlohy, plánovanie, notifikácie), obsahové dáta (dokumenty) a komunikačné dáta (chatové relácie a správy). Model kombinuje väzby jeden-ku-viacerým aj viacerí-ku-viacerým. Hierarchické vzťahy sme riešili priamo cudzími kľúčmi, zatiaľ čo flexibilné prepojenia (napr. štítky alebo závislosti úloh) sme realizovali cez asociačné tabuľky.

Životný cyklus údajov sme riadili politikami CASCADE a SET NULL. Pri zmazaní nadradeného záznamu sa závislé dáta odstránia tam, kde nemajú samostatný význam; v prípadoch, keď je potrebné zachovať históriu, sa referencia nastaví na NULL. Tým sme znížili riziko osirelých záznamov a zachovali auditovateľnosť.

Integritu sme doplnili obmedzeniami NOT NULL, UNIQUE a kompozitnými unikátnymi kľúčmi, ktoré bránia duplicitným väzbám v asociačných tabuľkách. Väčšina atribútov je relačná (identifikátory, referencie, stavy, časové značky), pričom JSON polia sme použili len tam, kde bola potrebná vyššia flexibilita, napríklad pre metadáta, práva alebo citácie odpovedí AI.

Celkovo sme model navrhli ako rozšíriteľný: jadrové tabuľky, asociačné tabuľky a konfiguračné entity umožňujú pridávať nové funkcie inkrementálne bez zásadných zásahov do existujúcej schémy.

6 Autentifikácia

Autentifikácia je prvým krokom, ktorým prechádza každý používateľ systému. Zabezpečuje, že k citlivým projektovým dátam a dokumentom môžu pristupovať len oprávnené osoby. Systém sme implementovali pomocou knižnice Flask-Login [7] s dôrazom na bezpečnosť prihlasovacieho procesu.

6.1 Registrácia

Registráciu nových používateľov sme zámerne obmedzili pozývacím kódom. Tento mechanizmus zabraňuje neautorizovaným osobám vytvoriť si konto a zabezpečuje, že do systému sa dostanú len skutočne pozvaní členovia tímu.

Postup registrácie:

- Používateľ otvorí registračný formulár a zadá meno, e-mailovú adresu, heslo a pozývací kód.
- Systém overí správnosť pozývacieho kódu (porovnaním s hodnotou `REGISTRATION_INVITE_` v konfigurácii).
- Heslo je zahashované pomocou funkcie `generate_password_hash` z Werkzeug [4] pred uložením do databázy. Pôvodné heslo sa nikdy neukladá.
- Po úspešnej registrácii je používateľovi pridelená predvolená rola a je presmerovaný na prihlasovanie (obr. `effig:registracia`).

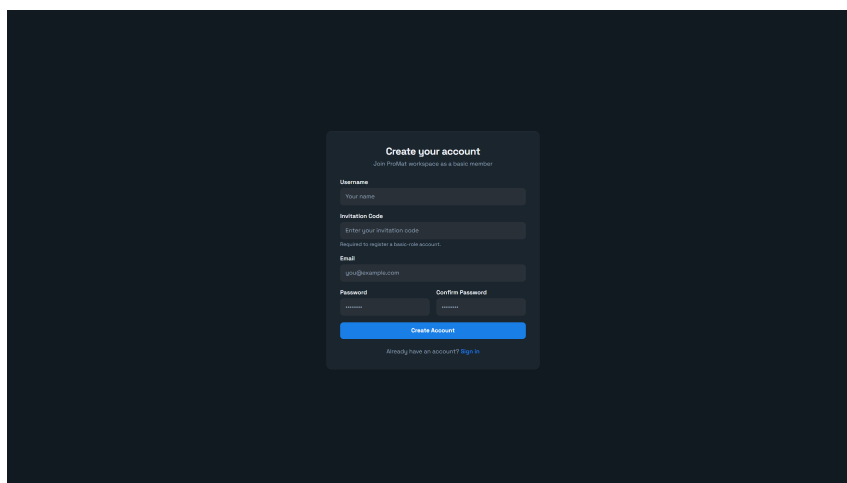


Figure 6.1 – Registračný formulár

6.2 Prihlásenie

Prihlasovací proces overuje totožnosť používateľ a porovnaním zadaných prihlasovacích údajov s uloženými hodnotami v databáze.

Postup prihlásenia:

- Používateľ zadá e-mailovú adresu a heslo do prihlasovacieho formulára.
- Flask-Login overí, či používateľ s daným e-mailom existuje v databáze.
- Werkzeug funkcia `check_password_hash` porovná zadané heslo s uloženým hashom.
- Pri úspešnom prihlásení Flask-Login vytvorí reláciu a uloží informáciu o prihlásenom používateľovi do HTTPS session cookie.
- Pri nesprávnych údajoch systém zobrazí chybové hlásenie bez prezradenia, ktorý z údajov je nesprávny (obr. `effig:prihlasenie`).

Systém obsahuje aj vstavaného administrátora, ktorého prihlasovacie údaje sme nakonfigurovali cez premenné prostredia (`BUILTIN_ADMIN_USERNAME`, `BUILTIN_ADMIN_EMAIL`, `BUILTIN_ADMIN_PASSWORD`). Tento administrátor sa vytvorí automaticky pri prvom spustení aplikácie.

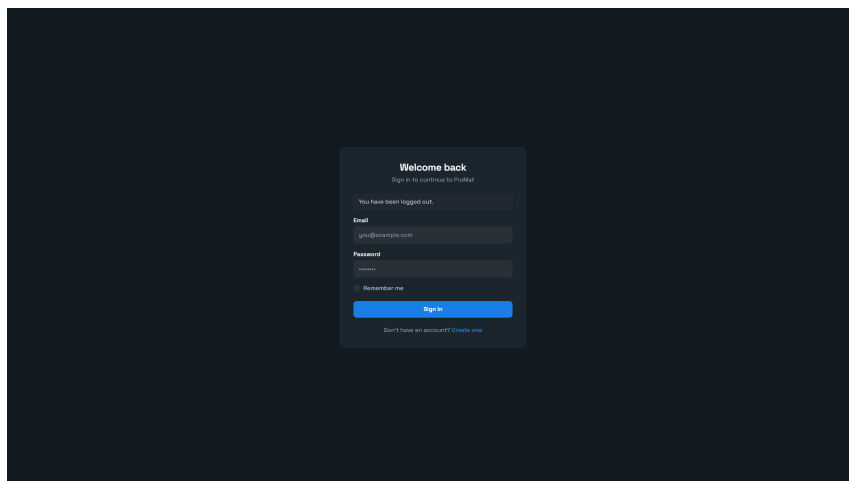


Figure 6.2 – Prihlasovací formulár

6.3 Správa relácie

Po úspešnom prihlásení Flask-Login udržiava reláciu používateľ a počas celej práce s aplikáciou.

Všetky chránené stránky (routes označené dekorátorom `@login_required`) automaticky presmerujú neprihlásených používateľov na prihlasovací formulár. Po prihlásení sú používatelia presmerovaní späť na pôvodne požadovanú stránku.

Odhlásenie prebieha manuálne cez odkaz v navigácii. Flask-Login vymaže reláciu a presmeruje používateľa na prihlasovací formulár. Na ochranu formulárov pred CSRF útokmi sme využili Flask-WTF [8], ktorý automaticky generuje a overuje CSRF token pri každom odoslaní formulára.

7 Projekty a úlohy

Jadrom aplikácie je správa projektov a úloh. V tejto kapitole opisujeme, ako sme navrhli vytváranie a organizovanie projektov, priradovanie úloh členom tímu a sledovanie ich stavu od vytvorenia až po dokončenie.

7.1 Správa projektov

Projekt je základnou organizačnou jednotkou systému. Každý projekt združuje súvisiace úlohy, dokumenty a členov tímu. Oprávnený používateľ (administrátor alebo manažér) môže vytvoriť nový projekt vyplnením formulára s nasledujúcimi atribútmi:

- **Názov** – krátky identifikačný názov projektu
- **Popis** – podrobnejší opis cieľov a obsahu projektu
- **Status** – aktuálny stav projektu (aktívny, dokončený, archivovaný)

Zoznam projektov je zobrazený na prehľadovej stránke s možnosťou filtrovania podľa statusu. Každý projekt je zobrazený ako karta s kľúčovými informáciami – názvom, progresom a poslednou aktualizáciou (obr. effig:projekty).

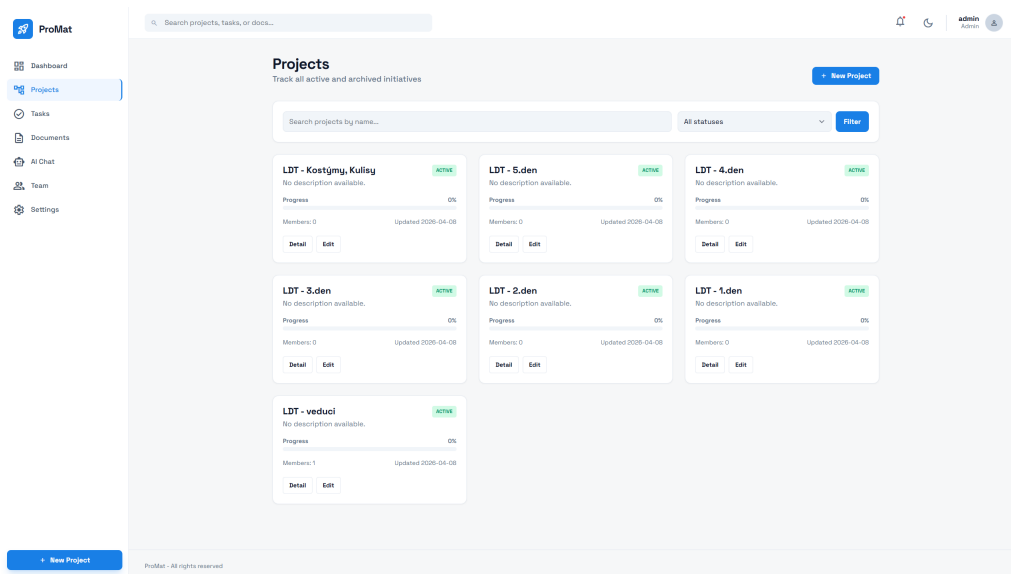


Figure 7.1 – Zoznam projektov

Detail projektu zobrazuje všetky úlohy, priradených členov, nahrané dokumenty a históriu aktivity (obr. effig:detail_projektu). Projekt je možné upravovať, odstrániť, alebo

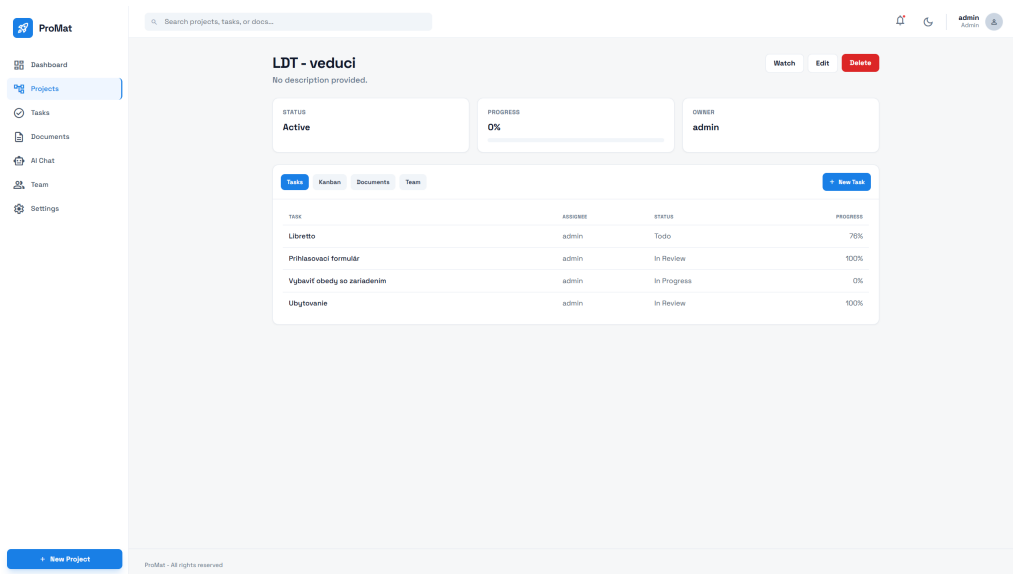


Figure 7.2 – Detail projektu

7.2 Správa úloh

Úlohy sme navrhli ako súčasť konkrétneho projektu. Pri vytváraní úlohy používateľ zadáva:

- **Názov a popis** – čo treba urobiť a prečo
- **Prioritu** – nízka, stredná alebo vysoká
- **Termín dokončenia** – dátum, do ktorého má byť úloha hotová
- **Status** – aktuálny stav (nová, v riešení, dokončená)
- **Priradenie** – člen tímu zodpovedný za splnenie úlohy

Úlohy sú zobrazené vo viacerých pohľadoch. Zoznamový pohľad umožňuje triedenie a filtrovanie podľa priority, termínu alebo priradeného člena. Kanban pohľad zobrazuje úlohy v stĺpcoch podľa stavu a umožňuje presúvanie úloh drag-and-drop interakciou priamo v prehliadači (obr. effig:kanban).

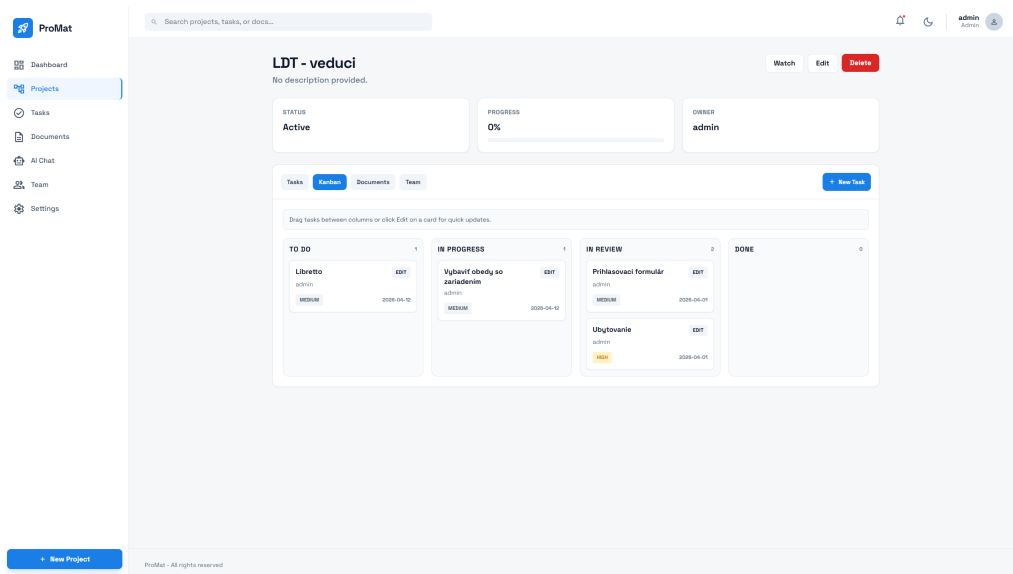


Figure 7.3 – Kanban tabuľ a úloh projektu

K úlohe je možné pridávať komentáre, položky na checklist a prikladat' dokumenty. Každá zmena stavu úlohy generuje upozornenie pre prideleného člena tímu.

7.3 Priradenie členov

Systém sme navrhli s podporou viacerých používateľov pracujúcich na jednom projekte. Administrátor alebo vlastník projektu môže prizvať ďalších registrovaných používateľov do projektu. Priradení členovia majú prístup k projektovým úlohám a dokumentom v súlade so svojou systémovou rolou (obr. effig:clenovia).

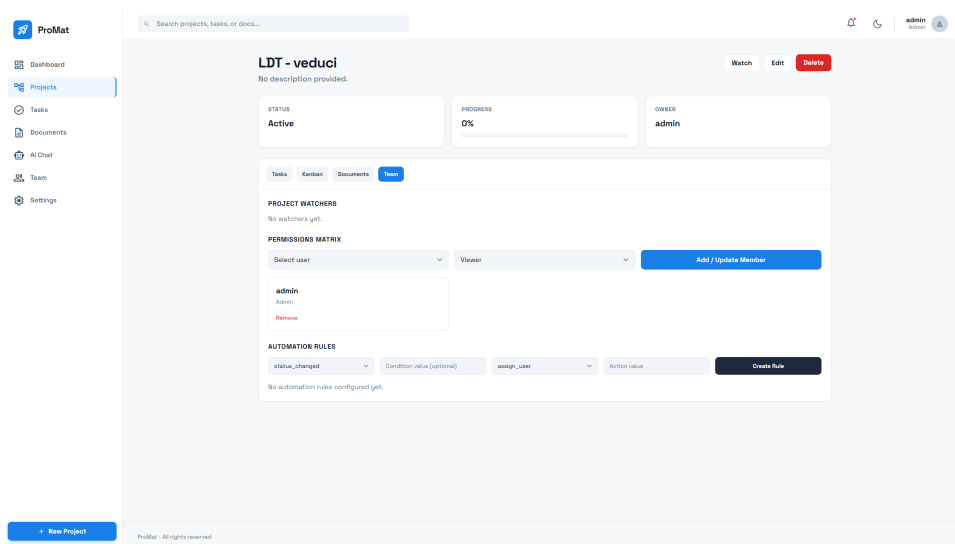


Figure 7.4 – Správa členov projektu

8 Dokumenty

Správa dokumentov je jednou z kľúčových funkcionalít systému. Implementovali sme nahrávanie, organizovanie a sprístupňovanie súborov priamo v kontexte projektov a úloh. Systém podporuje formáty PDF a DOCX, pričom z každého nahraného súboru automaticky extrahuje text pre potreby AI chatu.

8.1 Tvorba dokumentov

Nahrávanie dokumentu je dostupné z detailu projektu alebo úlohy. Používateľ vyberie súbor zo svojho zariadenia a systém vykoná nasledujúce kroky:

- **Validácia** – systém overí, či má súbor povolený formát (PDF alebo DOCX) a či neprekračuje maximálnu veľkosť 16 MB.
- **Uloženie** – súbor je uložený na lokálny filesystem servera do adresára `uploads/` s unikátnym názvom (UUID), aby sa zabránilo kolíziám mien.
- **Extrakcia textu** – z PDF súborov je text extrahovaný knižnicou PyPDF2 [13], z DOCX súborov knižnicou python-docx [12]. Extrahovaný text je uložený do stĺpca `extracted_text` v databáze.
- **Záznam v databáze** – vytvorí sa záznam `Document` s metadátami: pôvodný názov súboru, cesta k uloženému súboru, typ, projekt a nahrávateľ (obr. `effig:nahravanie`).

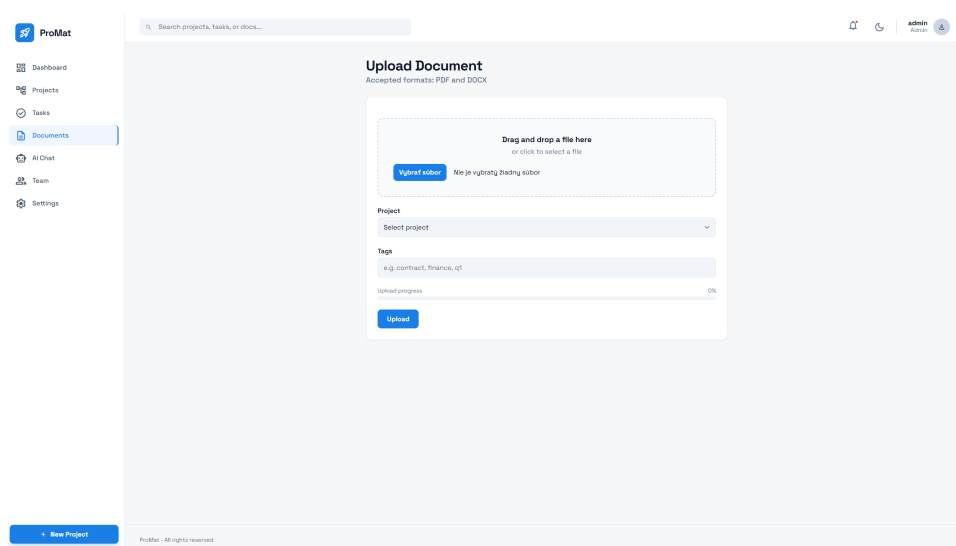


Figure 8.1 – Nahrávanie dokumentu

Nahrané dokumenty sú zobrazené v prehľadnom zozname s názvom, typom súboru a dátumom nahratia (obr. `effig:dokumenty`). Každý dokument je možné stiahnuť alebo otvoriť v AI chate na kladenie otázok nad jeho obsahom.

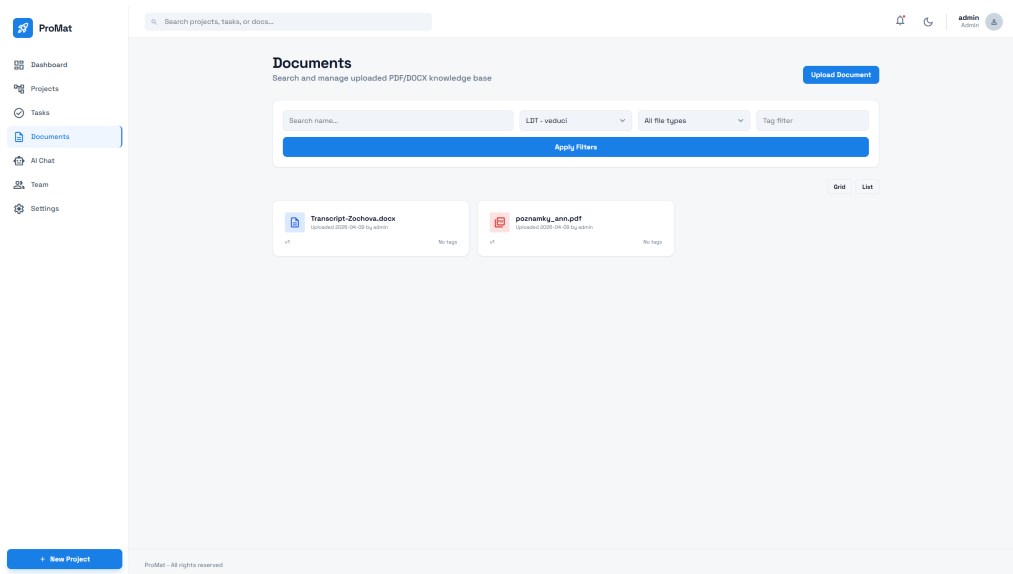


Figure 8.2 – Zoznam dokumentov projektu

8.2 Zdieľanie dokumentov

Dokumenty sú zdieľané v rámci projektu – všetci členovia projektu majú prístup k dokumentom nahraným v jeho kontexte. Tým sme zabezpečili tímovú spoluprácu bez nutnosti posielania súborov e-mailom alebo cez externé úložiská.

Obsah dokumentov neposielame na externé servery. Súbory sú uložené lokálne na serveri organizácie a AI model pristupuje k extrahovanému textu priamo z lokálnej databázy. Toto riešenie zaručuje, že citlivé informácie zostanú výhradne v infraštruktúre organizácie.

9 AI Chat

AI chat je najinovátnejšou súčast'ou systému. Umožňuje používateľom klásť otázky v prirodzenom jazyku nad obsahom nahraných dokumentov. Systém odpovie na základe extrahovaného textu z vybraných súborov, čím výrazne urýchľuje orientáciu v rozsiahlych podkladoch.

9.1 Integrácia AI

AI chat sme v aplikácii implementovali ako samostatnú komunikačnú vrstvu, ktorá pracuje nad vybraným kontextom z dokumentov, projektov a úloh. Backend sme postavili na lokálnom modeli cez službu Ollama [10], pričom predvolený model je `gemma4 : 2 6b` [16].

Z pohľadu ochrany údajov sme riešenie navrhli tak, aby zostalo čo najviac vnútri infraštruktúry systému. Otázky používateľ a sa spracúvajú lokálne a model pracuje s kontextom, ktorý je dostupný v databáze aplikácie. Pri odpovediach sa zohľadňuje aj história konkrétnej konverzácie, takže chat vie nadväzovať na predchádzajúce otázky bez nutnosti opakovať celý kontext.

9.2 Rozhranie chatu

Používateľské rozhranie AI chatu sme rozdelili na dva hlavné panely. V ľavej časti sa nachádza história konverzácií, kde si používateľ môže prepnúť medzi jednotlivými vláknami alebo založiť novú konverzáciu. Pravá časť obsahuje samotné správy, vstupné pole a ovládanie kontextu.

Nový chat je možné vytvoriť buď prázdny, alebo s pripojeným dokumentom. Po otvorení konverzácie sa správy zobrazujú v chronologickom poradí, pričom odpovede asistenta sú formátované pomocou Markdownu. Tým sa výsledný text stáva prehľadnejším a ľahšie čitateľným najmä pri odrážkach, odkazoch alebo kratších zhrnutiach (obr. `effig:ai_chat`).

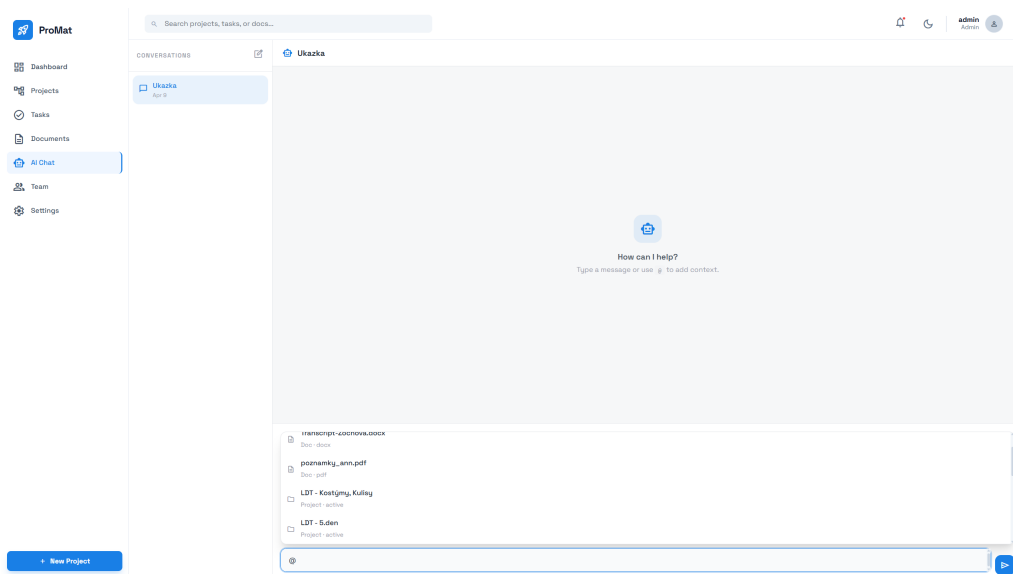


Figure 9.1 – Rozhraní AI chatu

Vstupné pole podporuje zadávanie zmienok pomocou znaku @. Po jeho zadaní sa zobrazí načítavač dokumentov, projektov a úloh, ktoré má používateľ k dispozícii. Po výbere položky sa vytvorí vizuálna značka a do správy sa doplní štruktúrovaná referencia, ktorú backend vie pri spracovaní preložiť na konkrétny záznam v databáze.

Samotné odoslanie správy prebieha cez asynchrónne volanie bez obnovovania stránky. Používateľ okamžite vidí svoju správu aj indikátor písania, následne sa pridá odpoveď modelu. Pri nedostupnosti AI backendu sa zobrazí chybové hlásenie, aby bolo správanie systému zrozumiteľné aj pri výpadku služby (obr. *effig:ai_odpoved*).

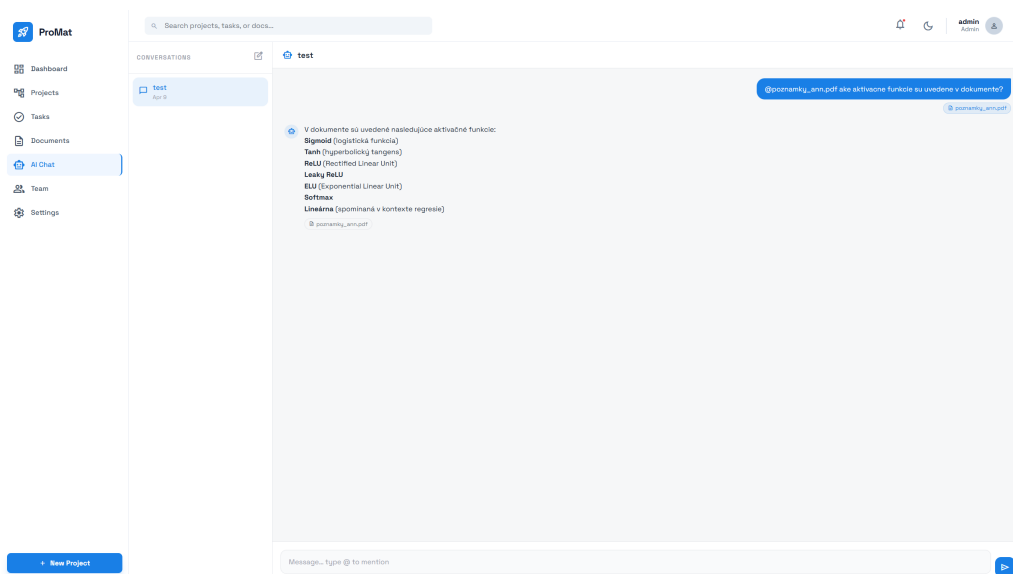


Figure 9.2 – Príklad AI odpovede nad obsahom dokumentu

Na obrázku je vidieť, že odpoveď asistenta vie vychádzať z pripojeného dokumentu a zároveň zachovávať kontext predchádzajúcej komunikácie. Tým sa AI chat stáva praktickým

nástrojom na rýchle prehľadávanie obsahu bez nutnosti manuálneho čítania celého súboru.

10 Dashboard

Dashboard sme navrhli ako centrálny ovládací panel aplikácie ProMat, ktorý poskytuje používateľom rýchly prehľad aktuálneho stavu ich projektov, úloh a tímov. Slúži ako vstupný bod po prihlásení a umožňuje rýchly prístup k kľúčovým metrikám a aktívnym projektom.

10.1 Prehľad a úvodné informácie

Hlavnú stránku dashboarda sme namapovali na endpoint `/dashboard`. Zobrazuje komplexný prehľad pracovného priestoru používateľa. V hornej časti sa zobrazujú tri kľúčové metriky:

- **Aktívne projekty** – počet aktívnych projektov, do ktorých má používateľ prístup,
- **Blížiac sa termíny** – počet úloh s termínom vypršania v nasledujúcich 7 dňoch,
- **Aktuálny dátum** – aktuálny dátum pre rýchlu orientáciu.

Tieto informácie sme zobrazili vo forme kariet v prehľadnom gride s vizuálnym odlíšením.

10.2 AI-asistované poznatky

Jedným z kľúčových prvkov dashboarda sú **AI Insights** – automaticky generované upozornenia a poznatky na základe aktuálneho stavu úloh a projektov. Systém dynamicky vytvára maximálne 4 poznatky naraz podľa priority.

10.2.1 Typy poznatkov

Medzi hlavné typy poznatkov patria:

- **Detekcia rizík** – upozornenie na úlohy po termíne,
- **Upozornenie na termín** – upozornenie na úlohy splatné v najbližších 24 hodinách,
- **Plánovaná práca** – prehľad plánovaných úloh v horizonte nasledujúceho týždňa,
- **Úspech** – zvýraznenie nedávno dokončených úloh,
- **Vyžaduje pozornosť** – signalizácia vysoko prioritných úloh bez stanoveného termínu.

Každý poznatok obsahuje popis, odkaz na podrobnosti a vizuálne odlíšenie farbou, ktorá indikuje typ upozornenia (obr. `effig:dashboardstatistiky`).

10.3 Sledovanie prioritných projektov

Sekciu sledovania prioritných projektov sme navrhli tak, aby zobrazovala top 3 aktívne projekty usporiadané podľa času poslednej úpravy. Pri každom projekte sa uvádza názov, percento pokroku, textové označenie termínu (napr. blíži sa termín alebo po termíne), farebný indikátor stavu a počet členov tímu.

Používateľ môže kliknúť na projekt a prejsť na stránku s jeho podrobnosťami.

10.4 Vyhľadávanie

Do aplikácie sme zaradili globálnu vyhľadávaciu funkciu dostupnú na endpoint `/search`. Používateľ môže rýchlo vyhľadávať projekty podľa názvu alebo popisu, úlohy podľa názvu alebo popisu a dokumenty podľa názvu alebo obsahu.

Vyhľadávanie je necitlivé na veľkosť písmen a aplikuje sa na projekty viditeľné danému používateľovi (na základe oprávnení a členstva v tímoch). Výsledky sú zoradené chronologicky podľa času vytvorenia alebo aktualizácie.

10.5 Bezpečnosť a oprávnenia

Všetky zobrazené údaje na dashboarde sme filtrovali podľa oprávnení používateľa. Funkcia `_visible_project_ids_query()` zabezpečuje, že používateľ vidí iba projekty, ku ktorým má oprávnený prístup. Systém automaticky aplikuje pravidlá členstva tímu a všetky metriky aj poznatky sa počítajú iba z viditeľných projektov a úloh (obr. `effig:dashboard_hla`).

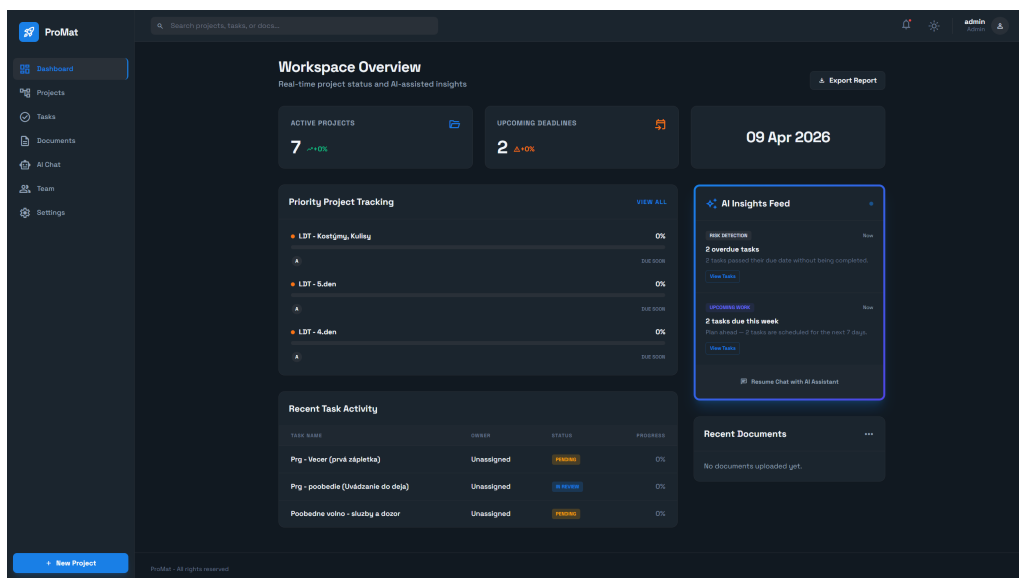


Figure 10.1 – Dashboard aplikácie ProMat

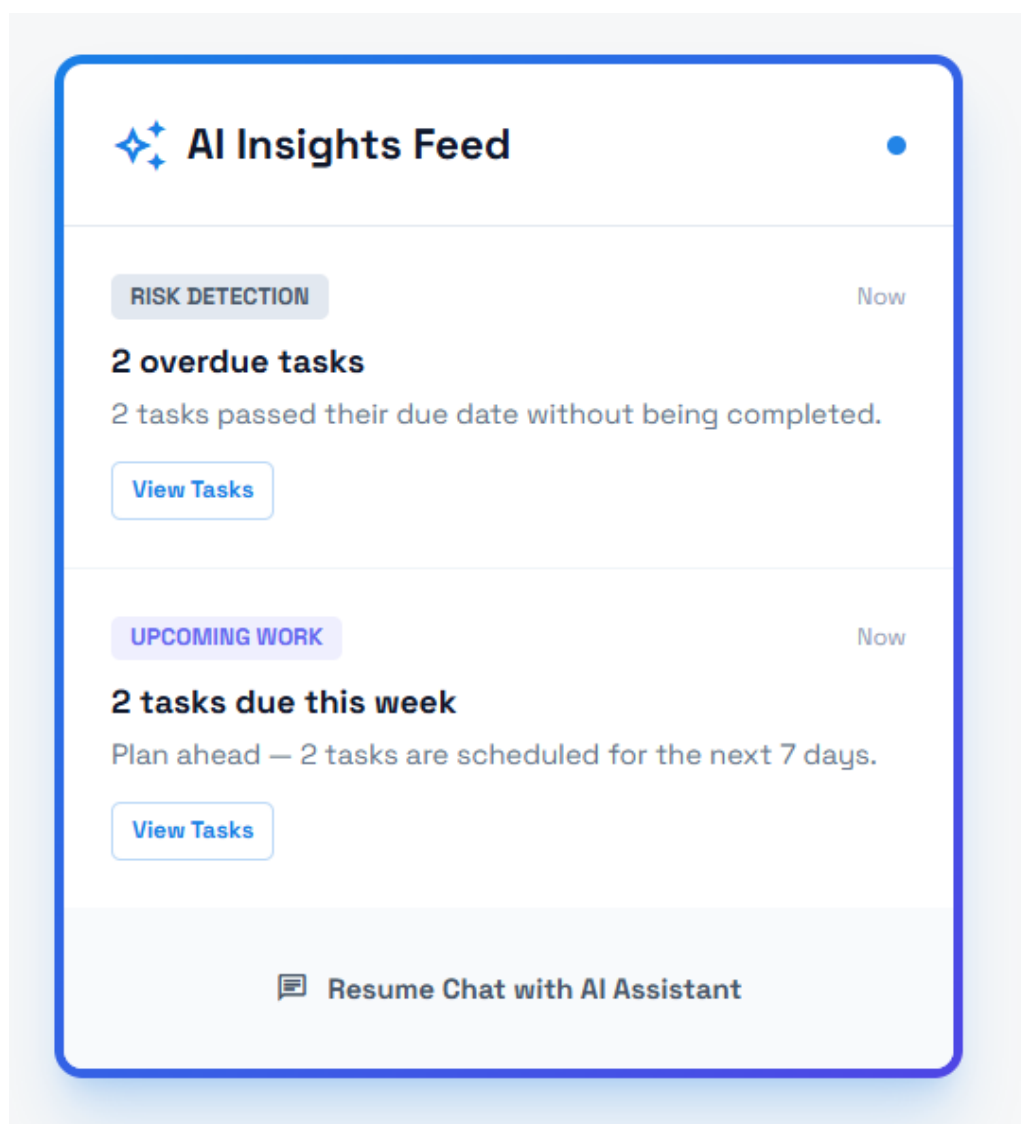


Figure 10.2 – Štatistiky a AI-asistované poznatky na dashboarde

Záver

Cieľom tejto práce bolo navrhnúť a implementovať webový systém na správu projektov s integrovaným AI chatom nad dokumentmi. Výsledná aplikácia ProMat naplňa všetky stanovené požiadavky a predstavuje funkčný nástroj pre tímy, ktoré potrebujú centralizovať riadenie projektov, úloh a dokumentácie na jednom mieste.

V teoretickej časti sme analyzovali problematiku roztrieštenej práce s nástrojmi, porovnali existujúce riešenia (napríklad Trello) a identifikovali ich kľúčové obmedzenia, predovšetkým chýbajúcu inteligentnú prácu s priloženými dokumentmi. Na konci tejto časti bol popísaný technologický zásobník zvoleného riešenia.

V praktickej časti sme opísali architektonický návrh systému, autentifikáciu, správu projektov a úloh vrátane Kanban tabuľe, nahrávanie a zdieľanie dokumentov, AI chat nad ich obsahom a informačný dashboard.

Najvýznamnejším prínosom projektu je integrácia lokálneho LLM servera Ollama s modelom Gemma 4, vďaka ktorej môžu tímy využívať výhody umelej inteligencie pri práci s citlivými dokumentmi bez nutnosti odosielania dát na externé cloudové servery. Toto riešenie je vhodné najmä pre organizácie s prísnejšími bezpečnostnými alebo legislatívnymi požiadavkami na ochranu dát.

Predkladaný projekt preukázal, že prepojenie projektového manažmentu s inteligentnou prácou s dokumentmi je technicky realizovateľné a prakticky prínosné riešenie, ktoré môže výrazne zvýšiť efektivitu tímov v rôznych typoch organizácií.

Pri tvorbe aplikácie sa zároveň potvrdilo, že moderné open-source technológie umožňujú vytvoriť robustné riešenie aj bez potreby komerčných platforiem. Kombinácia frameworku Flask, relačnej databázy a lokálneho LLM servera poskytla dostatočný výkon, modularitu a možnosť ďalšieho rozširovania podľa potrieb používateľov.

Priestor pre budúci rozvoj vidíme najmä v podpore integrácie s externými systémami a vo vylepšení AI chatu o presnejšie citovanie zdrojových pasáží z dokumentov.

Na základe dosiahnutých výsledkov možno konštatovať, že stanovený cieľ práce bol splnený v plnom rozsahu. Aplikácia ProMat predstavuje prakticky použiteľný základ, na ktorom je možné ďalej budovať komplexný interný nástroj pre projektové riadenie a efektívnu prácu s dokumentmi podporenú umelou inteligenciou.

ZOZNAM BIBLIOGRAFICKÝCH ODKAZOV

- [1] Atlassian. Trello – manage your team’s projects from anywhere. [online]. Atlassian. Dostupné na: <https://trello.com>, 2024. Navštívené: 8.4.2026.
- [2] Python Software Foundation. Python documentation. [online]. Python Software Foundation. Dostupné na: <https://docs.python.org>, 2025. Navštívené: 8.4.2026.
- [3] Armin Ronacher et al. Flask documentation. [online]. Pallets Projects. Dostupné na: <https://flask.palletsprojects.com>, 2024. Navštívené: 8.4.2026.
- [4] Armin Ronacher et al. Werkzeug documentation. [online]. Pallets Projects. Dostupné na: <https://werkzeug.palletsprojects.com>, 2024. Navštívené: 8.4.2026.
- [5] Armin Ronacher et al. Jinja documentation. [online]. Pallets Projects. Dostupné na: <https://jinja.palletsprojects.com>, 2024. Navštívené: 8.4.2026.
- [6] SQLAlchemy authors. SQLAlchemy documentation. [online]. SQLAlchemy. Dostupné na: <https://docs.sqlalchemy.org>, 2024. Navštívené: 8.4.2026.
- [7] Flask-Login contributors. Flask-login documentation. [online]. Flask-Login. Dostupné na: <https://flask-login.readthedocs.io>, 2024. Navštívené: 8.4.2026.
- [8] Flask-WTF contributors. Flask-wtf documentation. [online]. Flask-WTF. Dostupné na: <https://flask-wtf.readthedocs.io>, 2024. Navštívené: 8.4.2026.
- [9] Flask-Caching contributors. Flask-caching documentation. [online]. Flask-Caching. Dostupné na: <https://flask-caching.readthedocs.io>, 2024. Navštívené: 8.4.2026.
- [10] Ollama contributors. Ollama – get up and running with large language models. [online]. Ollama. Dostupné na: <https://ollama.com>, 2024. Navštívené: 8.4.2026.
- [11] Google. Google gen ai python sdk documentation. [online]. Google. Dostupné na: <https://googleapis.github.io/python-genai/>, 2025. Navštívené: 8.4.2026.

- [12] Steve Canny. python-docx documentation. [online]. python-docx. Dostupné na: <https://python-docx.readthedocs.io>, 2024. Navštívené: 8.4.2026.
- [13] pypdf contributors. pypdf documentation. [online]. pypdf. Dostupné na: <https://pypdf.readthedocs.io>, 2024. Navštívené: 8.4.2026.
- [14] Saurabh Kumar. python-dotenv documentation. [online]. python-dotenv. Dostupné na: <https://saurabh-kumar.com/python-dotenv/>, 2024. Navštívené: 8.4.2026.
- [15] D. Richard Hipp. Sqlite documentation. [online]. SQLite Consortium. Dostupné na: <https://www.sqlite.org/docs.html>, 2024. Navštívené: 8.4.2026.
- [16] Google DeepMind. Gemma 4 technical report. [online]. Google DeepMind. Dostupné na: <https://ai.google.dev/gemma/docs/core>, 2025. Navštívené: 8.4.2026.
- [17] Git contributors. Git reference manual. [online]. Git. Dostupné na: <https://git-scm.com/doc>, 2024. Navštívené: 8.4.2026.